

Chapitre 1 : INTRODUCTION AUX SYSTEMES D'EXPLOITATION

Objectifs : Ce que vous allez apprendre à la fin de ce chapitre :

A) Objectifs Généraux :

- Comprendre le rôle central des systèmes d'exploitation dans le fonctionnement des ordinateurs.
- Identifier et expliquer les composants principaux d'un système d'exploitation.
- Explorer l'évolution historique des systèmes d'exploitation, des débuts de l'informatique aux systèmes modernes.

B) Objectifs Spécifiques :

1. Définition et Rôle des Systèmes d'Exploitation

- Décrire ce qu'est un système d'exploitation et ses fonctions principales.
- Illustrer le lien entre l'utilisateur, le matériel et les applications.

2. Évolution Historique

- Reconstituer les étapes majeures de l'évolution des systèmes d'exploitation.
- Identifier les avancées technologiques clés (exemple : multitâche, interfaces graphiques).

3. Composants d'un Système d'Exploitation

- Analyser les fonctions principales du noyau, des interfaces utilisateur, et de la gestion des périphériques, des fichiers, et des processus.
- Donner des exemples pratiques pour chaque composant dans des contextes réels.

4. Typologie des Systèmes d'Exploitation

- Différencier les types de systèmes d'exploitation (monotâche, multitâche, temps réel, distribués, etc.).
- Associer chaque type à des exemples concrets et à des cas d'utilisation.

5. Objectifs des Systèmes d'Exploitation

- Décrire les objectifs fondamentaux d'un OS : gestion des ressources, sécurité, convivialité, et performance.
- Expliquer comment ces objectifs influencent la conception des systèmes modernes.

C) Compétences Attendues : À la fin de ce chapitre, l'étudiant devrait être capable de :

- Décrire les fonctions principales d'un système d'exploitation.
- Identifier les composants essentiels et leur rôle dans le fonctionnement global.
- Relier les types de systèmes d'exploitation à des contextes d'utilisation spécifiques.
- Expliquer pourquoi et comment les systèmes d'exploitation ont évolué au fil du temps.

1. Introduction

Les systèmes d'exploitation (SE) jouent un rôle fondamental dans le fonctionnement des ordinateurs modernes. En agissant comme une interface entre l'utilisateur et le matériel, ils assurent une gestion efficace des ressources et permettent l'exécution des applications. Ce chapitre introduit les concepts essentiels liés aux systèmes d'exploitation, leur évolution historique, leurs composants, leurs différents types, ainsi que leurs principaux objectifs. Cette exploration fournit une compréhension de leur importance dans le monde informatique, où ils sont au cœur de l'interaction entre les utilisateurs et la technologie.

2. Notion du Système Informatique : SI

Un **système informatique** est un ensemble de composants (ressources **logicielles** et **matérielles**) interconnectés qui travaillent ensemble pour collecter, traiter, stocker et distribuer des informations pour satisfaire les besoins informatiques d'un (es) utilisateur(s). Il constitue la base des environnements numériques modernes et repose sur l'interaction de plusieurs éléments essentiels.

Un système informatique ne fonctionne de manière optimale que si ces composants sont bien intégrés, permettant ainsi d'assurer la performance, la fiabilité et la sécurité des opérations numériques.

On peut diviser grossièrement un système informatique en quatre composants : *le Matériel*, *le Système d'exploitation*, *les Programmes d'application* et *les Utilisateurs* (voir figure 1.1).

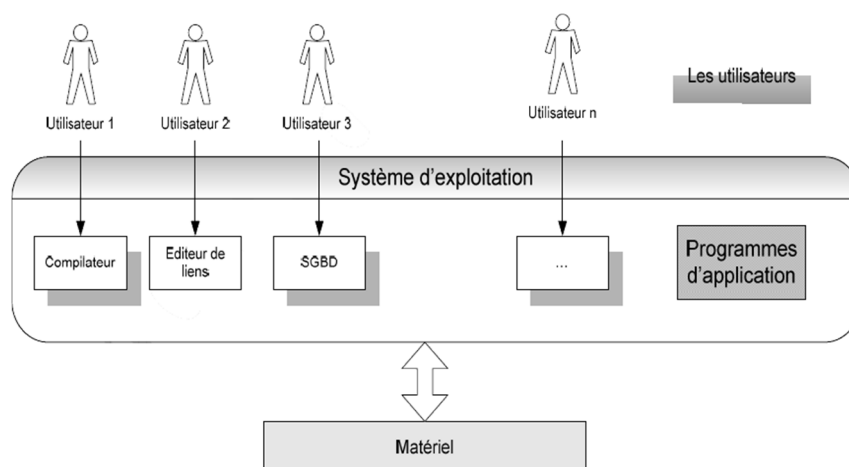


Figure 1.1 Vue abstraite d'un Système Informatique

- **Le matériel (Hardware):** Ensemble des ressources physiques nécessaires à la résolution informatique d'un problème. Il fournit les ressources physiques de base, tels que : le **processeur** qui exécute les instructions, la **mémoire centrale** qui sauvegarde les données et les instructions à exécuter, la **mémoire secondaire** qui sauvegarde les informations, ainsi que **les périphériques d'Entrées/Sorties** (clavier, souris, écran, modem, scanner, etc.).
- **Les programmes d'application :** Ensemble de ressources logicielles qui définissent les façons d'utiliser les ressources matérielles pour résoudre les problèmes informatiques des utilisateurs. Exemples : Compilateurs, Systèmes de gestion de base de données (SGBD), Applications de gestion, Logiciels de Bureautique, ... etc.
- **Les utilisateurs :** Il peut exister plusieurs types d'utilisateurs (des personnes, des machines, des robots, d'autres ordinateurs, etc.) qui interagissent avec le système et le matériel à travers les programmes d'application.
- **Le système d'exploitation :** Programme qui contrôle et coordonne l'utilisation du matériel parmi les divers programmes d'application pour les divers utilisateurs. Le SE est donc une partie importante de tout système informatique.

3. Qu'est-ce qu'un Système d'Exploitation ?

Un système d'exploitation (SE ou **O.S.** en anglais '**Operating System**') est un logiciel essentiel qui agit comme une interface ou intermédiaire entre l'utilisateur (Applications) d'un ordinateur et le matériel de celui-ci.

Le but principal d'un système d'exploitation est de fournir un **environnement convivial** qui permet à un utilisateur d'exécuter des programmes sans faire des erreurs en assurant une **transparence** entre l'utilisateur et les détails de la machine. Sans SE, l'utilisation d'un ordinateur serait extrêmement complexe et inefficace (voire impossible).

Il doit satisfaire, les deux fonctions suivantes :

- **Allouer et partager** les ressources entre plusieurs utilisateurs simultanément,
- **Contrôler et vérifier l'exécution** des programmes utilisateurs pour éviter les erreurs et l'utilisation incorrecte de l'ordinateur.

En conclusion, on pourrait dire que :

Un système d'exploitation = Programme d'allocation + Programme de contrôle.

4. Rôles d'un Système d'Exploitation

Les rôles d'un système d'exploitation sont divers :

- **Gestion de processeur** : le SE doit gérer l'allocation du processeur aux différents programmes (applications) pouvant s'exécuter. Cette allocation se fait grâce à un **algorithme d'ordonnancement** qui planifie l'exécution des programmes (voir chapitre 2).
- **Gestion de la mémoire centrale** : le SE est chargé de gérer l'espace mémoire alloué à chaque application. En cas d'insuffisance de mémoire physique, le système d'exploitation peut créer une zone mémoire sur le disque dur, appelée «**mémoire virtuelle**» (voir chapitre 4).
- **Gestion des entrées/sorties** : le SE doit gérer l'accès aux périphériques, c'est-à-dire faire la liaison entre les appels de haut niveau des programmes utilisateurs (Exemple ***getchar()** en java*) et les opérations de bas niveau de l'unité d'échange responsable du périphérique (unité d'échange, ici c'est le clavier) : c'est le pilote d'entrées-sorties (**Driver**) qui assure cette correspondance.
 - **Les périphériques d'entrée** : Ces unités permettent à l'utilisateur d'entrer des données, des commandes et des programmes qui seront gérés par l'unité de traitement, exemple : **Le clavier, La souris, le scanner, la carte magnétique, le crayon optique, l'écran tactile** et les **mémoires auxiliaires** permettent également d'entrer l'information.
 - **Les périphériques de sortie** : Ces unités permettent à l'utilisateur de *recevoir des résultats* des calculs ou des manipulations de données effectués par l'ordinateur, exemple : **L'écran, L'imprimante**, D'autres organes tels que les **haut-parleurs, la table traçante** et les **mémoires auxiliaires** permettent également de transmettre de l'information.
- **Gestion de la concurrence** : Comme plusieurs programmes coexistent en mémoire centrale, ceux-ci peuvent vouloir communiquer pour échanger des données. Par ailleurs, il faut synchroniser l'accès aux données partagées afin de maintenir leur cohérence. Le système offre des outils de communication et de synchronisation entre programmes (Voir module SE2).
- **Gestion des droits** : le SE est chargé de la sécurité liée à l'exécution des programmes en garantissant que les ressources ne sont utilisées que par les programmes et utilisateurs possédant les droits adéquats.
- **Gestion des fichiers** : le SE gère la lecture et l'écriture dans le système de fichiers et les droits d'accès aux fichiers par les utilisateurs et les applications.

5. Composantes d'un système d'exploitation

Un système d'exploitation est une structure complexe organisée en plusieurs composants, chacun ayant des rôles spécifiques pour garantir le fonctionnement optimal du système. Voici les principaux composants d'un SE avec des exemples concrets pour chaque cas.

5.1. Le noyau (en anglais *kernel*) : Le noyau est le cœur du système d'exploitation. Il fonctionne en arrière-plan pour gérer les ressources matérielles et coordonner l'exécution des processus. Il représente les fonctions fondamentales du système d'exploitation telles que la gestion de la mémoire, des processus, des fichiers, des entrées-sorties principales, et des fonctionnalités de communication.

- **Rôles principaux :**
 - Planification des processus : Le noyau décide quels processus utilisent le processeur et pour combien de temps.
 - Gestion de la mémoire : Alloue et libère de la mémoire pour les applications.
 - Gestion des périphériques : Contrôle les interactions entre le matériel et les pilotes.
- **Exemple :** Lorsqu'un utilisateur demande à enregistrer un fichier, le noyau coordonne l'accès au disque dur en envoyant des commandes spécifiques via le pilote du disque, tout en s'assurant que les données ne corrompent pas d'autres processus en cours.

5.2. L'interpréteur de commande (en anglais *shell*, ou *Coquille* en français, par opposition au noyau) : Il permet la communication avec le système d'exploitation par l'intermédiaire d'un langage de commandes, afin de permettre à l'utilisateur de piloter les périphériques en ignorant toutes les caractéristiques du matériel qu'il utilise (*Voir partie TP*). Il existe deux types principaux d'interfaces :

- Interface en ligne de commande (CLI) : Les utilisateurs interagissent en tapant des commandes textuelles.
Exemple : Dans Linux, un utilisateur peut exécuter la commande `ls` pour afficher les fichiers d'un répertoire.
- Interface graphique (GUI) : Permet l'interaction via des éléments visuels comme des icônes, des fenêtres et des menus.
Exemple : Le bureau Windows où un utilisateur peut ouvrir un fichier en double-cliquant sur son icône.

5.3. Le système de fichiers (en anglais «file system», noté FS) :

Les systèmes de fichiers permettent d'organiser et de stocker des données sur des supports comme les disques durs, les SSD ou les clés USB. Ils définissent comment les données sont organisées, nommées et sécurisées.

Fonctions principales (Voir partie TP) :

1. Organisation des données : Arborescence de répertoires et fichiers.
2. Accès sécurisé : Gestion des permissions (lecture, écriture, exécution).
3. Gestion de la redondance : Certains systèmes comme ZFS préviennent les corruptions de données.

Exemple :

- Sous Windows, le système de fichiers NTFS offre des permissions de sécurité granulaires.
- Sous Linux, ext4 est courant pour ses performances et sa fiabilité.
- Les systèmes FAT32 sont utilisés sur les clés USB pour leur compatibilité avec divers appareils.

6. Historique et évolution des SE**6.1. Les premiers systèmes (Système Porte ouverte) 1945-1955 :**

Les premiers ordinateurs étaient physiquement des machines très volumineuses utilisées à partir d'une console. Le programme était d'abord chargé manuellement en mémoire, à partir de bandes perforées et en utilisant des interrupteurs. Le programmeur/opérateur pouvait surveiller l'exécution du programme par les voyants d'affichage de la console.

À cette époque, il n'existait pas de véritables systèmes d'exploitation. Les ordinateurs étaient programmés directement en modifiant le câblage ou en utilisant des cartes perforées.

Exemple : L'ENIAC (Electronic Numerical Integrator and Computer) nécessitait une reconfiguration manuelle pour chaque programme.

Caractéristiques : Traitement séquentiel, absence d'automatisation, interaction directe avec le matériel.

6.2. Systèmes simples de traitements par lots 1955-1965

Avec l'arrivée des premiers systèmes d'exploitation rudimentaires, les tâches étaient regroupées en lots (batch processing). Les programmes étaient exécutés sans intervention humaine entre chaque tâche.

Exemple : Le système IBM 7090 utilisait des cartes perforées pour exécuter des tâches automatiquement.

Caractéristiques : Automatisation partielle, files d'attente de tâches, exécution séquentielle.

6.3. Systèmes multi-programmés et interactifs 1965-1980

L'introduction des interfaces graphiques a révolutionné l'utilisation des ordinateurs personnels. Cette période a vu l'émergence des systèmes d'exploitation grand public avec des interfaces conviviales.

Exemple : Windows 3.1, Mac OS, qui ont introduit l'utilisation des icônes, fenêtres et souris.

Caractéristiques : Accessibilité accrue, gestion simplifiée des fichiers, développement d'applications utilisateur.

6.4. Systèmes modernes et mobiles 1995 - aujourd'hui

Les systèmes d'exploitation modernes offrent une connectivité avancée, des fonctionnalités de sécurité renforcées et une compatibilité avec de multiples plateformes, y compris les appareils mobiles.

Exemple : Windows 10/11, macOS, Android, iOS.

Caractéristiques : Cloud computing, intelligence artificielle, interfaces tactiles, optimisation des performances.

7. Types de Systèmes d'Exploitation

Les systèmes d'exploitation (SE) se déclinent en plusieurs types en fonction des besoins et des environnements où ils sont déployés. Voici les principaux types de SE accompagnés d'exemples :

7.1. Systèmes d'exploitation mono-utilisateur et mono-tâche

Ces SE permettent à un seul utilisateur d'exécuter une tâche à la fois. Ce sont généralement des systèmes très simples, adaptés aux appareils nécessitant peu de calculs complexes.

Exemple : MS-DOS (Microsoft Disk Operating System) : Utilisé sur les premiers PC, ce système ne pouvait exécuter qu'une seule commande ou programme à la fois, comme afficher une liste de fichiers ou ouvrir un fichier texte.

7.2. Systèmes d'exploitation mono-utilisateur et multitâche

Ces SE permettent à un seul utilisateur d'exécuter plusieurs tâches simultanément. Ils sont couramment utilisés sur les ordinateurs personnels modernes.

Exemples :

- **Windows 10/11** : Permet d'exécuter plusieurs applications, comme un traitement de texte, un navigateur Web et un lecteur de musique, en parallèle.
- **macOS** : Fournit un environnement multitâche fluide avec des fonctionnalités avancées comme Mission Control pour gérer plusieurs fenêtres.

7.3. Systèmes d'exploitation multiutilisateur

Ces SE permettent à plusieurs utilisateurs d'accéder à un seul système simultanément, en partageant les ressources de manière équitable.

Exemples :

- **Unix/Linux** : Supportent plusieurs utilisateurs connectés via des sessions locales ou à distance (par exemple, SSH). Un serveur Linux peut gérer des centaines d'utilisateurs exécutant des applications ou des scripts en même temps.
- **Mainframes IBM** : Utilisés dans les grandes entreprises pour exécuter des tâches critiques avec plusieurs utilisateurs connectés.

7.4. Systèmes d'exploitation temps réel (RTOS)

Les RTOS sont conçus pour exécuter des tâches dans un délai précis et prédéfini, ce qui est crucial pour les systèmes embarqués et les applications critiques.

Exemples :

- **VxWorks** : Utilisé dans l'aéronautique, les satellites et les robots industriels.
- **FreeRTOS** : Couramment utilisé dans les dispositifs IoT (Internet des Objets) pour gérer des capteurs ou des microcontrôleurs.
- **Systèmes embarqués dans les voitures modernes** : Comme ceux utilisés pour les airbags, où la réponse doit être quasi instantanée.

7.5. Systèmes d'exploitation distribués

Ces SE gèrent un réseau d'ordinateurs interconnectés comme s'il s'agissait d'un seul système. Ils partagent les ressources et les tâches entre plusieurs machines.

Exemples :

- **Apache Hadoop** : Plateforme utilisée pour le traitement de grandes quantités de données dans des environnements distribués.
- **Google File System (GFS)** : SE distribué utilisé par Google pour gérer ses vastes centres de données.

7.6. Systèmes d'exploitation embarqués

Ces SE sont intégrés dans des appareils spécifiques pour exécuter des tâches dédiées. Ils sont souvent optimisés pour consommer peu de ressources.

Exemples :

- **SE des Smart TVs** : Tizen OS de Samsung ou WebOS de LG.
- **Dispositifs médicaux** : Pacemakers ou moniteurs de santé.
- **SE des consoles de jeu** : Nintendo Switch utilise un système basé sur FreeBSD.

7.7. Systèmes d'exploitation mobiles

Ces SE sont conçus pour les smartphones et les tablettes. Ils optimisent l'utilisation de l'énergie et les interactions tactiles.

Exemples :

- **Android** : Basé sur Linux, il équipe une majorité des smartphones dans le monde.
- **iOS** : Utilisé par les appareils Apple tels que l'iPhone et l'iPad.

7.8. Systèmes d'exploitation réseau

Ces SE permettent la gestion des ressources partagées et des communications dans un réseau.

Exemples :

- **Windows Server** : Utilisé dans les entreprises pour gérer les fichiers partagés, les serveurs Web, et les services de messagerie.

- **Linux (avec Samba)** : Utilisé pour configurer des serveurs de fichiers et d'impression dans des réseaux locaux.

Chaque type de système d'exploitation est conçu pour répondre à des exigences spécifiques. Cette diversité permet d'adapter les SE à des environnements variés, des simples ordinateurs personnels aux systèmes critiques complexes.

8. Objectifs d'un Système d'Exploitation

Le système d'exploitation (SE) joue un rôle crucial dans le fonctionnement d'un ordinateur. Ses objectifs principaux peuvent être divisés en plusieurs catégories, chacun visant à améliorer l'efficacité, la sécurité et l'expérience utilisateur.

8.1. Gestion efficace des ressources

L'un des principaux objectifs d'un SE est de gérer efficacement les ressources matérielles et logicielles de l'ordinateur (processeur, mémoire, stockage, périphériques, etc.).

Exemples :

- **Gestion du processeur** : Le SE utilise des algorithmes de planification (comme Round-Robin ou Shortest Job Next) pour allouer le processeur aux processus en cours d'exécution.
 - **Exemple concret** : Lorsqu'un utilisateur exécute simultanément un lecteur vidéo et un navigateur Web, le SE attribue du temps processeur à chacun pour garantir une expérience fluide.
- **Gestion de la mémoire** : Le SE décide quelles parties de la mémoire sont utilisées par quels programmes et gère les situations de mémoire insuffisante avec des mécanismes comme la pagination ou la mémoire virtuelle.
 - **Exemple** : Windows utilise un fichier d'échange sur le disque dur lorsque la mémoire vive (RAM) est pleine.

8.2. Fournir une interface utilisateur

Un autre objectif clé est de permettre aux utilisateurs d'interagir avec le matériel de manière simple et intuitive. Cette interaction peut se faire via :

- **Ligne de commande (CLI)** : Les utilisateurs avancés peuvent entrer des commandes textuelles pour interagir directement avec le système.
 - **Exemple** : Sur Linux, la commande `cp source.txt destination.txt` permet de copier un fichier.
- **Interface graphique (GUI)** : Les utilisateurs peuvent interagir avec le système à l'aide d'éléments visuels comme des fenêtres, des menus et des icônes.
 - **Exemple** : Sur macOS, les utilisateurs peuvent glisser-déposer des fichiers entre des dossiers.

8.3. Exécution de programmes

Le SE fournit un environnement stable pour l'exécution des applications, garantissant leur isolation et leur performance. Il gère :

- Le chargement des programmes en mémoire.
- La protection des processus pour éviter qu'un programme n'interfère avec un autre.

Exemple : Lorsqu'un utilisateur joue à un jeu vidéo, le SE veille à ce que le jeu reçoive une priorité suffisante pour éviter des ralentissements dus à d'autres processus.

8.4. Sécurité et contrôle d'accès

Le SE protège le système contre les accès non autorisés et garantit la confidentialité et l'intégrité des données.

- **Contrôle des utilisateurs** : Le SE impose des politiques de permissions (lecture, écriture, exécution).
 - **Exemple** : Sous Linux, seuls les utilisateurs autorisés peuvent modifier les fichiers système.
- **Protection contre les virus et logiciels malveillants** : Certains SE intègrent des mécanismes pour limiter les actions des logiciels malveillants.
 - **Exemple** : Windows Defender analyse les fichiers pour détecter les menaces potentielles.

8.5. Fiabilité et récupération après panne

Un bon SE est conçu pour être fiable et capable de se rétablir rapidement en cas de panne, tout en minimisant les pertes de données.

Exemples :

- **Sauvegarde automatique** : macOS offre une fonctionnalité appelée Time Machine pour sauvegarder régulièrement les données de l'utilisateur.
- **Systèmes de fichiers journalisés** : Ext4 (sous Linux) garde une trace des opérations en cours pour éviter la perte de données en cas de coupure de courant.

8.6. Support pour la communication entre processus

Le SE facilite la communication entre différents processus (Inter-Process Communication, IPC), essentielle pour les applications complexes.

Exemple :

- Sous Linux, deux processus peuvent utiliser des **tuyaux (pipes)** pour échanger des données en temps réel. Par exemple, la commande `ls | grep ".txt"` liste tous les fichiers puis filtre ceux contenant ".txt".

8.7. Optimisation de la performance

Un objectif majeur du SE est de maximiser les performances du système tout en utilisant efficacement les ressources disponibles.

Exemples :

- **Mécanismes de cache** : Le SE utilise la mémoire cache pour accélérer l'accès aux données fréquemment utilisées.
 - **Exemple concret** : Windows précharge les fichiers système critiques en RAM pour accélérer le démarrage.
- **Planification des entrées/sorties** : Le SE optimise les opérations d'accès au disque pour réduire les temps d'attente.

8.8. Compatibilité et portabilité

Un SE vise à offrir un environnement compatible pour exécuter des logiciels sur différentes configurations matérielles. De plus, certains SE sont conçus pour être portables et fonctionner sur diverses plateformes matérielles.

Exemples :

- **Compatibilité** : Windows permet d'exécuter d'anciennes applications grâce à son mode de compatibilité.
- **Portabilité** : Le noyau Linux est utilisé sur une large gamme de matériels, des serveurs aux smartphones.

8.9. Support de la connectivité réseau

Le SE facilite la mise en réseau et l'accès aux ressources distantes. Il inclut des outils pour gérer les connexions, les protocoles et la sécurité réseau.

Exemple :

- Sous Windows, la fonctionnalité **Remote Desktop** permet de se connecter à distance à un autre ordinateur pour effectuer des tâches.

Ces objectifs définissent le rôle fondamental d'un SE et expliquent son importance pour les systèmes informatiques modernes. Leur réalisation permet d'offrir une expérience utilisateur optimale, même dans des environnements complexes.

9. Conclusion

Les systèmes d'exploitation sont bien plus qu'un simple logiciel : ils constituent l'épine dorsale de l'informatique moderne. Leur capacité à gérer des ressources complexes, à garantir la sécurité, à offrir des interfaces intuitives et à répondre à des besoins variés témoigne de leur indispensable rôle dans les environnements informatiques. En comprenant

leur architecture, leur évolution, leurs types et leurs objectifs, on peut mieux appréhender la manière dont ces systèmes facilitent l'utilisation des ordinateurs tout en permettant des avancées technologiques majeures. Cette base solide est essentielle pour approfondir davantage l'étude des systèmes d'exploitation et explorer leurs applications dans différents contextes.